
2

Nombres con sentido

por Tim Ottinger



Introducción

En el *software*, los nombres son omnipresentes. Aparecen en variables,

funciones, argumentos, clases y paquetes. Asignamos nombres a archivos y a directorios, a archivos jar, war y ear. Usamos nombres constantemente. Por ello, debemos hacerlo bien. A continuación, veremos algunas reglas básicas para crear nombres correctos.

Usar nombres que revelen las intenciones

Es fácil afirmar que los nombres deben revelar nuestras intenciones. Lo que queremos recalcar es la *importancia* de hacerlo. Elegir nombres correctos lleva tiempo, pero también ahorra trabajo. Por ello, preste atención a los nombres y cámbielos cuando encuentre otros mejores. Todo el que lea su código se lo agradecerá.

El nombre de una variable, función o clase debe responder una serie de cuestiones básicas. Debe indicar por qué existe, qué hace y cómo se usa. Si un nombre requiere un comentario, significa que no revela su cometido.

```
int d; // tiempo transcurrido en días
```

El nombre `d` no revela nada. No evoca una sensación de tiempo transcurrido, ni de días. Debe elegir un nombre que especifique lo que se mide y la unidad de dicha medida:

```
int elapsedTimeInDays;  
int daysSinceCreation;  
int daysSinceModification;  
int fileAgeInDays;
```

La elección de nombres que revelen intenciones facilita considerablemente la comprensión y la modificación del código. ¿Para qué sirve el siguiente código?

```
public List<int[]> getThem() {  
    List<int[]> list1 = new ArrayList<int[]>();  
    for (int[] x : theList)  
        if (x[0] == 4)  
            list1.add(x);  
    return list1;  
}
```

¿Por qué es complicado saber la función de este código? No hay expresiones complejas. Los espacios y el sangrado son razonables. Sólo hay tres variables y dos constantes. Ni siquiera contiene clases complejas o

métodos polimórficos, sólo una lista de matrices (o eso parece).

El problema no es la simplicidad del código sino su carácter *implícito*: el grado en el que el contexto no es explícito en el propio código. Implícitamente, el código requiere que sepamos las respuestas a las siguientes preguntas:

- ¿Qué contiene theList?
- ¿Qué significado tiene el subíndice cero de un elemento de theList?
- ¿Qué importancia tiene el valor 4?
- ¿Cómo se usa la lista devuelta?

Las respuestas a estas preguntas no se encuentran en el código, pero se podrían haber incluido. Imagine que trabaja en un juego de buscar minas. El tablero es una lista de celdas llamada theList. Cambiemos el nombre por gameBoard.

Cada celda del teclado se representa por medio de una matriz. El subíndice cero es la ubicación de un valor de estado que, cuando es 4, significa que se ha detectado. Al asignar nombres a estos conceptos mejoramos considerablemente el código:

```
public List<int[]> getFlaggedCells() {  
    List<int[]> flaggedCells = new ArrayList<int[]>();  
    for (int[] cell : gameBoard)  
        if (cell[STATUS_VALUE] == FLAGGED)  
            flaggedCells.add(cell);  
    return flaggedCells;  
}
```

La simplicidad del código no ha cambiado. Sigue teniendo los mismos operadores y constantes y el mismo número de niveles anidados, pero ahora es mucho más explícito. Podemos crear una sencilla clase para celdas en lugar de usar una matriz de elementos int. Puede incluir una función que revele el objetivo (con el nombre isFlagged) para ocultar los números. El resultado es una nueva versión de la función:

```
public List<Cell> getFlaggedCells() {  
    List<Cell> flaggedCells = new ArrayList<Cell>();  
    for (Cell cell : gameBoard)  
        if (cell.isFlagged())  
            flaggedCells.add(cell);  
}
```

```
    return flaggedCells;  
}
```

Con estos sencillos cambios de nombre, es fácil saber qué sucede. Es la ventaja de seleccionar nombres adecuados.

Evitar la desinformación

Los programadores deben evitar dejar pistas falsas que dificulten el significado del código. Debemos evitar palabras cuyo significado se aleje del que pretendemos. Por ejemplo, `hp`, `aix` y `sco` son nombres de variables pobres ya que son los nombres de plataformas o variantes de Unix. Aunque se trate del código de una hipotenusa y `hp` parezca la abreviatura correcta, puede no serlo.

No haga referencia a un grupo de cuentas como `accountList` a menos que realmente sea una lista (`List`). La palabra lista tiene un significado concreto para los programadores. Si el contenedor de las cuentas no es realmente una lista, puede provocar falsas conclusiones^[6]. Por tanto, resulta más adecuado usar `accountGroup`, `bunchOfAccounts` o simplemente `accounts`.

Evite usar nombres con variaciones mínimas. ¿Cuánto se tarda en apreciar la sutil diferencia entre `XYZControllerForEfficientHandlingOfStrings` y `XYZControllerForEfficientStorageOfStrings` en un módulo? Ambas palabras tienen una forma similar.

La ortografía similar de conceptos parecidos es información; el uso de ortografía incoherente es desinformación. En los entornos modernos de Java, el código se completa de forma automática. Escribimos varios caracteres de un nombre y pulsamos varias teclas para obtener una lista de posibles opciones de un nombre. Es muy útil si los nombres de elementos similares se ordenan alfabéticamente de forma conjunta y si las diferencias son muy evidentes, ya que es probable que el programador elija un objeto por nombre sin fijarse en los comentarios o la lista de métodos proporcionados por una clase.

Un ejemplo de nombre desinformativo sería el uso de la `l` minúscula o la `o` mayúscula como nombres de variables, sobre todo combinados. El problema, evidentemente, es que se parecen a las constantes `1` y `0` respectivamente:

```
int a = 1;
if ( 0 == 1 )
    a = 01;
else
    1 = 01;
```

El lector puede pensar que es una invención, pero hemos visto código con abundancia de estos elementos. En un caso, el autor del código, sugirió usar una fuente distinta para que las diferencias fueran más evidentes, una solución que se hubiera transmitido a todos los futuros programadores como tradición oral o en un documento escrito. El problema se resolvió con carácter definitivo y sin necesidad de crear nuevos productos, con tan sólo cambiar los nombres.

Realizar distinciones con sentido

Los programadores se crean un problema al crear código únicamente dirigido a un compilador o intérprete. Por ejemplo, como se puede usar el mismo nombre para hacer referencia a dos elementos distintos en el

mismo ámbito, puede verse tentado a cambiar un nombre de forma arbitraria. En ocasiones se hace escribiéndolo incorrectamente, lo que provoca que los errores ortográficos impidan la compilación^[7].



No basta con añadir series de números o palabras adicionales, aunque eso satisfaga al compilador. Si los nombres tienen que ser distintos, también deben tener un significado diferente.

Los nombres de series numéricas (`a1`, `a2`... `aN`) son lo contrario a los nombres intencionados. No desinforman, simplemente no ofrecen información; son una pista sobre la intención del autor. Fíjese en lo siguiente:

```
public static void copyChars(char a1[], char a2[]) {
    for (int i = 0; i < a1.length; i++) {
        a2[i] = a1[i];
    }
}
```

Esta función se lee mejor cuando se usa *source* y *destination* como nombres de argumentos.

Las palabras adicionales son otra distinción sin sentido. Imagine que tiene la clase *Product*. Si tiene otra clase con el nombre *ProductInfo* o *ProductData*, habrá creado nombres distintos, pero con el mismo significado. *Info* y *Data* son palabras adicionales, como *a*, *an* y *the*.

No es incorrecto usar prefijos como *a* y *the* mientras la distinción tenga sentido. Imagine que usa *a* para variables locales y *for* para argumentos de funciones^[8]. El problema aparece cuando decide invocar la variable *theZork* porque ya tiene otra variable con el nombre *zork*.

Las palabras adicionales son redundantes. La palabra *variable* no debe incluirse nunca en el nombre de una variable. La palabra *table* no debe incluirse nunca en el nombre de una tabla. ¿Es mejor *NameString* que *Name*? ¿Podría ser *Name* un número de coma flotante? En caso afirmativo, incumple la regla anterior sobre desinformación. Imagine que encuentra una clase con el nombre *Customer* y otra con el nombre *CustomerObject*. ¿Cuál sería la distinción? ¿Cuál representa mejor el historial de pagos de un cliente?

Existe una aplicación que lo ilustra. Hemos cambiado los nombres para proteger al culpable. Veamos el error exacto:

```
getActiveAccount();
getActiveAccounts();
getActiveAccountInfo();
```

¿Cómo saben los programadores de este proyecto qué función deben invocar?

En ausencia de convenciones concretas, la variable *moneyAmount* no se distingue de *money*, *customerInfo* no se distingue de *customer*, *accountData* no se distingue de *account* y *theMessage* no se distingue de *message*. Debe diferenciar los nombres de forma que el lector aprecie las diferencias.

Usar nombres que se puedan pronunciar

A los humanos se nos dan bien las palabras. Gran parte de nuestro cerebro se dedica al concepto de palabras. Y, por definición, las palabras son pronunciables. Sería una pena malgastar esa parte de nuestro cerebro dedicada al lenguaje hablado. Por tanto, cree nombres pronunciables. Si no lo puede pronunciar, no podrá explicarlo sin parecer tonto. Es un factor importante, ya que la programación es una actividad social.

Conozco una empresa que usa genymdhms (fecha de generación, año, mes, día, hora, minuto y segundo) y lo pronuncian tal cual. Yo tengo la costumbre de pronunciar todo tal y como lo veo escrito, de forma que muchos analistas y diseñadores acabaron por llamarme algo como «genimedemes». Era un chiste y nos parecía divertido, pero en realidad estábamos tolerando el uso de nombres pobres. Teníamos que explicar las variables a los nuevos programadores y cuando las pronunciaban, usaban palabras inventadas en lugar de nombres correctos. Compare:

```
class DtaRcrd102 {
    private Date genymdhms;
    private Date modymdhms;
    private final String pszqint = "102";
    /*... */
};
```

con:

```
class Customer {
    private Date generationTimestamp;
    private Date modificationTimestamp;
    private final String recordId = "102";
    /*... */
};
```

Ahora se puede mantener una conversación inteligente: «Eh, Mikey, fíjate en este registro. La marca de tiempo de generación es para mañana. ¿Cómo es posible?»

Usar nombres que se puedan buscar

Los nombres de una letra y las constantes numéricas tienen un problema: no son fáciles de localizar en el texto. Se puede detectar `MAX_CLASSES_PER_STUDENT`, pero el número 7 resulta más complicado. Las búsquedas pueden devolver el dígito como parte de nombres de archivo, otras definiciones de constantes o expresiones en las que se use con otra intención. Mucho peor si la constante es un número extenso y alguien ha intercambiado los dígitos, lo que genera un error inmediato y no aparece en la búsqueda.

Del mismo modo, el nombre `e` es una opción muy pobre para variables que el programador tenga que buscar. Es la letra más usada en inglés y aparece en la práctica totalidad de los textos de un programa. A este respecto, los nombres extensos superan a los breves y cualquier nombre que se pueda buscar supera a una constante en el código.

Personalmente prefiero nombres de una letra que sólo se puedan usar como variables locales dentro de métodos breves. *La longitud de un nombre debe corresponderse al tamaño de su ámbito* [N5]. Si una variable o constante se usa en varios puntos del código, debe asignarle un nombre que se pueda buscar. Compare:

```
for (int j=0; j<34; j++) {  
    s += (t[j]*4)/5;  
}
```

con:

```
int realDaysPerIdealDay = 4;  
const int WORK_DAYS_PER_WEEK = 5;  
int sum = 0;  
for (int j = 0; j < NUMBER_OF_TASKS; j++) {  
    int realTaskDays = taskEstimate[j] * realDaysPerIdealDay;  
    int realTaskWeeks = (realdays / WORK_DAYS_PER_WEEK);  
    sum += realTaskWeeks;  
}
```

En este ejemplo, `sum` no es un nombre especialmente útil, pero al menos se puede buscar. Se usa una función más extensa, pero comprobará que resulta mucho más fácil buscar `WORK_DAYS_PER_WEEK` que todas las instancias de 5 y filtrar la lista a los casos con el significado adecuado.

Evitar codificaciones

Ya tenemos suficientes codificaciones como para tener que añadir otras nuevas. Al codificar información de tipos o ámbitos en un nombre se dificulta la descodificación. No parece razonable que todos los nuevos empleados tengan que aprender otro lenguaje de codificación además del código con el que van a trabajar. Es una carga mental innecesaria a la hora de intentar solucionar un problema. Los nombres codificados resultan impronunciables y suelen escribirse de forma incorrecta.

Notación húngara

Antiguamente, cuando trabajábamos con lenguajes en los que la longitud de los nombres era un reto, incumplíamos esta regla a nuestro pesar. Fortran forzaba las codificaciones convirtiendo la primera letra de un tipo en código. En sus primeras versiones, BASIC sólo se permitía una letra y un dígito. La notación húngara (HN) lo llevó a un nuevo nivel.

HN se consideraba muy importante en el API C de Windows, donde todo era un control entero, un puntero long, un puntero void o una de varias implementaciones de string (con diferentes usos y atributos). Por aquel entonces, el compilador no comprobaba los tipos, de modo que los programadores tenían que recordarlos.

En los lenguajes modernos disponemos de sistemas de tipos más completos y los compiladores recuerdan los tipos y los aplican. Es más, existe una tendencia a usar clases y funciones más breves para que los usuarios aprecien la declaración de las variables que usan.

Los programadores de Java no tienen que codificar tipos. Los objetos son de tipos fuertes y los entornos de edición han avanzado tanto que detectan un error de tipo antes de ejecutar la compilación. Por ello, en la actualidad HN y otras formas de codificación de tipos no son más que un impedimento. Hacen que sea más complicado cambiar el nombre o el tipo de una variable o clase. Dificultan la legibilidad del código y pueden hacer que el sistema de codificación confunda al lector:

```
PhoneNumber phoneString;  
// el nombre no cambia cuando cambia el tipo
```

Prefijos de miembros

Tampoco es necesario añadir `m_` como prefijo a los nombres de variables. Las clases y funciones tienen el tamaño necesario para no tener que hacerlo, y debe usar un entorno de edición que resalte o coloree los miembros para distinguirlos.

```
public class Part {  
    private String m_dsc; // La descripción textual  
    void setName(String name) {  
        m_dsc = name;  
    }  
}  
  
public class Part {  
    String description;  
    void setDescription(String description) {  
        this.description = description;  
    }  
}
```

Además, los usuarios aprenden rápidamente a ignorar el prefijo (o sufijo) y fijarse en la parte con sentido del nombre. Cuanto más código leemos, menos nos fijamos en los prefijos. En última instancia, los prefijos son un indicio de código antiguo.

Interfaces e Implementaciones

Existe un caso especial para usar codificaciones. Imagine por ejemplo que crea una factoría abstracta para crear formas. Esta factoría será una interfaz y se implementará por medio de una clase concreta. ¿Qué nombres debe asignar? ¿`IShapeFactory` y `ShapeFactory`? Prefiero las interfaces sin adornos. La `I` inicial, tan habitual en los archivos de legado actuales es, en el mejor de los casos, una distracción, y en el peor, un exceso de información. No quiero que mis usuarios sepan que se trata de una interfaz, solamente que se trata de `ShapeFactory`. Si tengo que codificar la interfaz o la implementación, opto por ésta última. Es mejor usar `ShapeFactoryImp` o incluso `CShapeFactory`, que codificar la interfaz.

Evitar asignaciones mentales

Los lectores no tienen que traducir mentalmente sus nombres en otros que ya conocen. Este problema suele aparecer al elegir entre no usar términos de dominio de problemas o de soluciones.

Es un problema de los nombres de variables de una sola letra. Un contador de bucles se podría bautizar como *i*, *j* o *k* (pero nunca *l*) si su ámbito es muy reducido y no hay conflictos con otros nombres, ya que los nombres de una letra son tradicionales en contadores de bucles. Sin embargo, en otros contextos, un nombre de una letra es una opción muy pobre: es como un marcador de posición que el lector debe asignar mentalmente a un concepto real. No hay peor motivo para usar el nombre *c* que *a* y *b* ya estén seleccionados.

Por lo general, los programadores son gente inteligente. A la gente inteligente le gusta presumir de sus habilidades mentales. Si puede recordar que *r* es la versión en minúscula de una URL sin el host y el sistema, debe ser muy listo.

Una diferencia entre un programador inteligente y un programador profesional es que este último sabe que la *claridad es lo que importa*. Los profesionales usan sus poderes para hacer el bien y crean código que otros puedan entender.

Nombres de clases

Las clases y los objetos deben tener nombres o frases de nombre como *Customer*, *WikiPage*, *Account* y *AddressParser*. Evite palabras como *Manager*, *Processor*, *Data*, o *Info* en el nombre de una clase. El nombre de una clase no debe ser un verbo.

Nombres de métodos

Los métodos deben tener nombres de verbo como `postPayment`, `deletePage` o `save`. Los métodos de acceso, de modificación y los predicados deben tener como nombre su valor y usar como prefijo `get`, `set` e `is` de acuerdo al estándar de `javabeans`^[9].

```
string name = employee.getName();  
customer.setName("mike");  
if (paycheck.isPosted())...
```

Al sobrecargar constructores, use métodos de factoría estáticos con nombres que describan los argumentos. Por ejemplo:

```
Complex fulcrumPoint = Complex.FromRealNumber(23.0);
```

es mejor que:

```
Complex fulcrumPoint = new Complex(23.0);
```

Refuerce su uso convirtiendo en privados sus constructores correspondientes.

No se exceda con el atractivo

Si los nombres son demasiado inteligentes, sólo los recordarán los que compartan el sentido del humor de su autor, y sólo mientras se acuerden del chiste. ¿Sabrán qué significa la función `HolyHandGrenade`?

Sin duda es atractiva, pero en este caso puede que `DeleteItems` fuera más indicado. Opte por la claridad antes que por el entretenimiento. En el código, el atractivo suele aparecer como formas coloquiales o jergas. Por ejemplo, no use `whack()` en lugar de `kill()`. No recurra a bromas culturales como `eatMyShorts()` si quiere decir `abort()`.



Diga lo que piense. Piense lo que diga.

Una palabra por concepto

Elija una palabra por cada concepto abstracto y manténgala. Por ejemplo,

resulta confuso usar `fetch`, `retrieve` y `get` como métodos equivalentes de clases distintas. ¿Cómo va a recordar qué método se corresponde a cada clase? Desafortunadamente, tendrá que recordar qué empresa, grupo o individuo ha creado la biblioteca o clase en cuestión para recordar qué término se ha empleado. En caso contrario, perderá mucho tiempo buscando en encabezados y fragmentos de código.

Los entornos de edición modernos como Eclipse e IntelliJ ofrecen pistas sensibles al contexto, como la lista de métodos que puede invocar en un determinado objeto. Pero esta lista no suele incluir los comentarios de nombres de funciones y listas de parámetros. Tendrá suerte si muestra los nombres de parámetros de las declaraciones de funciones. Los nombres de funciones deben ser independientes y coherentes para que pueda elegir el método correcto sin necesidad de búsquedas adicionales.

Del mismo modo, resulta confuso tener un controlador, un administrador y un control en la misma base de código. ¿Cuál es la diferencia entre `DeviceManager` y `ProtocolController`? ¿Por qué no son los dos controladores o administradores? ¿Son controladores? El nombre hace que espere que dos objetos tengan un tipo diferente y clases diferentes.

Un léxico coherente es una gran ventaja para los programadores que tengan que usar su código.

No haga juegos de palabras

Evite usar la misma palabra con dos fines distintos. Suele hacerse en juegos de palabras. Si aplica la regla de una palabra por conceptos, acabará con muchas clases que por ejemplo tengan un método `add`. Mientras las listas de parámetros y los valores devueltos de los distintos métodos `add` sean semánticamente equivalentes, no hay problema.

Sin embargo, alguien puede decidir usar la palabra `add` por motivos de coherencia, aunque no sea en el mismo sentido. Imagine que hay varias clases en las que `add` crea un nuevo valor sumando o concatenando dos valores existentes. Imagine ahora que crea una nueva clase con un método

que añada su parámetro a una colección. ¿Este método debe tener el método `add`? Parece coherente ya que hay otros muchos métodos `add`, pero en este caso hay una diferencia semántica, de modo que debemos usar un nombre como `insert` o `append`. Llamar `add` al nuevo método sería un juego de palabras.

Nuestro objetivo, como autores, es facilitar la comprensión del código. Queremos que el código sea algo rápido, no un estudio exhaustivo. Queremos usar un modelo en el que el autor sea el responsable de transmitir el significado, no un modelo académico que exija investigar el significado mostrado.

Usar nombres de dominios de soluciones

Recuerde que los lectores de su código serán programadores. Por ello, use términos informáticos, algoritmos, nombres de patrones, términos matemáticos y demás. No conviene extraer todos los nombres del dominio de problemas ya que no queremos que nuestros colegas tengan que preguntar el significado de cada nombre en especial cuando ya conocen el concepto bajo otro nombre diferente.

El nombre `AccountVisitor` tiene mucho significado para un programador familiarizado con el patrón `VISITOR`. ¿Qué programador no sabe lo que es `JobQueue`? Hay cientos de cosas técnicas que los programadores tienen que hacer y elegir nombres técnicos para dichas cosas suele ser lo más adecuado.

Usar nombres de dominios de problemas

Cuando no exista un término de programación para lo que esté haciendo, use el nombre del dominio de problemas. Al menos el programador que mantenga su código podrá preguntar el significado a un experto en dominios.

Separar los conceptos de dominio de soluciones y de problemas es parte del trabajo de un buen programador y diseñador. El código que tenga más relación con los conceptos del dominio de problemas tendrá nombres

extraídos de dicho dominio.

Añadir contexto con sentido

Algunos nombres tienen significado por sí mismos, pero la mayoría no. Por ello, debe incluirlos en un contexto, en clases, funciones y espacios de nombres con nombres adecuados. Cuando todo lo demás falle, pueden usarse prefijos como último recurso.

Imagine que tiene las variables `firstName`, `lastName`, `street`, `houseNumber`, `city`, `state` y `zipcode`. Si las combina, es evidente que forman una dirección. Pero si la variable `state` se usa de forma aislada en un método, ¿sabría que forma parte de una dirección? Puede añadir contexto por medio de prefijos: `addrFirstName`, `addrLastName`, `addrState`, etc. Al menos los lectores comprenderán que estas variables forman parte de una estructura mayor. Evidentemente, es mejor crear la clase `Address`. De ese modo, incluso el compilador sabrá que las variables pertenecen a un concepto más amplio.

Fíjese en el método del Listado 2-1. ¿Las variables necesitan un contexto con más sentido? El nombre de la función sólo ofrece parte del contexto, el resto se obtiene del algoritmo. Tras leer la función, verá que las tres variables `number`, `verb` y `pluralModifier` forman parte del mensaje `guess statistics`. Desafortunadamente, es necesario inferir el contexto. Al leer el método, el significado de las variables no es evidente.

Listado 2-1

Variables en un contexto ambiguo.

```
private void printGuessStatistics(char candidate, int count) {  
    String number;  
    String verb;  
    String pluralModifier;  
    if (count == 0) {  
        number = "no";  
        verb = "are";  
        pluralModifier = "s";  
    } else if (count == 1) {  
        number = "1";  
    }  
}
```

```

        verb = "is";
        pluralModifier = "";
    } else {
        number = Integer.toString(count);
        verb = "are";
        pluralModifier = "s";
    }
    String guessMessage = String.format(
        "There %s %s %s%s", verb, number, candidate, pluralModifier
    );
    print(guessMessage);
}

```

La función es demasiado extensa y las variables aparecen por todas partes. Para dividir la función en fragmentos más reducidos necesitamos crear una clase `GuessStatisticsMessage` y convertir a las tres variables en campos de la misma. De este modo contamos con un contexto más obvio para las tres variables. Forman parte sin duda de `GuessStatisticsMessage`. La mejora del contexto también permite que el algoritmo sea más limpio y se divida en funciones más reducidas (véase el Listado 2-2).

Listado 2-2

Variables con un contexto.

```

public class GuessStatisticsMessage {
    private String number;
    private String verb;
    private String pluralModifier;

    public String make(char candidate, int count) {
        createPluralDependentMessageParts(count);
        return String.format(
            "There %s %s %s%s",
            verb, number, candidate, pluralModifier);
    }

    private void createPluralDependentMessageParts(int count) {
        if (count == 0) {
            thereAreNoLetters();
        } else if (count == 1) {
            thereIsOneLetter();
        } else {
            thereAreManyLetters(count);
        }
    }

    private void thereAreManyLetters(int count) {
        number = "1";
    }
}

```

```

        verb = "is";
        pluralModifier = "";
    }

    private void thereIsOneLetter() {
        number = "1";
        verb = "is";
        pluralModifier = "";
    }

    private void thereAreNoLetters() {
        number = "no";
        verb = "are";
        pluralModifier = "s";
    }
}

```

No añadir contextos innecesarios

En la aplicación imaginaria Gas Station Deluxe, no es aconsejable usar el prefijo GSD en todas las clases. Es trabajar contra las herramientas proporcionadas. Introduzca G y pulse la tecla de finalización para acceder a una lista interminable de todas las clases del sistema. ¿Es lo correcto? ¿Por qué dificultar la ayuda del IDE?

Del mismo modo, imagine que ha creado la clase `MailingAddress` en un módulo de contabilidad de GSD, con el nombre `GSDAccountAddress`. Después, necesita una dirección de correo para la aplicación de contacto con el cliente. ¿Usará `GSDAccountAddress`? ¿Le parece el nombre correcto? 10 de los 17 caracteres son redundantes o irrelevantes.

Los nombres breves suelen ser más adecuados que los extensos, siempre que sean claros. No añada más contexto del necesario a un nombre. Los nombres `accountAddress` y `customerAddress` son perfectos para instancias de la clase `Address` pero no sirven como nombres de clase. `Address` sirve como nombre de clase. Para distinguir entre direcciones MAC, direcciones de puertos y direcciones Web, podría usar `PostalAddress`, `MAC` y `URI`. Los nombres resultantes son más precisos, el objetivo de cualquier nombre.

Conclusión

Lo más complicado a la hora de elegir un buen nombre es que requiere habilidad descriptiva y acervo cultural. Es un problema de formación más que técnico, empresarial o administrativo. Como resultado, mucha gente del sector no aprende a hacerlo bien.

La gente teme que al cambiar los nombres otros programadores se quejen. Nosotros no compartimos ese temor y agradecemos los cambios de nombre (siempre que sean a mejor). En muchos casos no memorizamos los nombres de clases y métodos. Usamos herramientas modernas para estos detalles y así poder centrarnos en si el código se lee como frases o párrafos, o al menos como tablas y estructuras de datos (una frase no siempre es la mejor forma de mostrar datos). Seguramente acabará sorprendiendo a alguien cuando cambie los nombres, como puede suceder con cualquier otra mejora del código. No deje que le detenga.

Aplice estas reglas y compruebe si mejora o no la legibilidad de su código. Si es el encargado de mantener código de terceros, use herramientas para solucionar estos problemas. Obtendrá recompensas a corto y largo plazo.